PROCESADORES DEL LENGUAJE

GRADO EN INGENIERÍA INFORMÁTICA

MEMORIA DE LA PRÁCTICA

Práctica 2 — Generación del analizador sintáctico o parser

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	Marco García
NIA:	100525997
Correo electrónico:	100525997@alumnos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Grado en Ingeniería Informática

Nombre:	Pablo Roig
NIA:	100525980
Correo electrónico:	100525980@alumnos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Grado en Ingeniería Informática

1. Introducción y objetivos

Esta memoria documenta el diseño e implementación del analizador sintáctico correspondiente a la segunda entrega de la práctica de Procesadores del Lenguaje. El objetivo de esta fase es construir, sobre el analizador léxico entregado en P1, un parser capaz de determinar si una secuencia de tokens pertenece al lenguaje Lava tal y como queda definido en el enunciado.

Para su implementación se ha empleado la librería ply.yacc, que genera tablas LALR(1) a partir de las reglas de producción definidas en Python. La gramática resultante es capaz de reconocer el lenguaje completo: tipos básicos, registros, expresiones aritméticas y lógicas con precedencia correcta, control de flujo (if-else, while, do-while), funciones con y sin parámetros, y todos los casos límite descritos en la especificación.

Además de reconocer la sintaxis general del lenguaje, la solución incorpora una capa adicional de validación estructural orientada a P2 para rechazar ciertos programas que, aunque podrían resultar gramaticalmente válidos con una gramática demasiado permisiva, el enunciado declara expresamente inválidos. En concreto se comprueban: la validez del lado izquierdo de una asignación, el contexto de uso de break, el contexto de uso de return y la presencia obligatoria de retorno en funciones no void.

En esta entrega no se realiza análisis semántico completo de tipos, tablas de símbolos ni conversiones automáticas; esas tareas quedan reservadas para P3.

2. Estructura general de la solución

La implementación se ha dividido en tres ficheros principales:

- **lexer.py:** define el conjunto de tokens del lenguaje, las expresiones regulares asociadas y la función build_lexer() que construye y devuelve el analizador léxico.
- **parser.py:** define la gramática del lenguaje mediante reglas de producción de PLY, establece la tabla de precedencia de operadores, construye una representación estructural intermedia (nodos) y realiza validaciones adicionales sobre el resultado del parseo.
- **main.py:** actúa como punto de entrada del programa. Gestiona la lectura del fichero de entrada, el modo de ejecución normal (parser completo) y el modo opcional --token (solo lexer).

Adicionalmente, el subdirectorio tests/ contiene una batería de 22 casos de prueba organizados en válidos e inválidos, junto con un script run_tests.py que automatiza su ejecución.

La visión de conjunto es la siguiente: el lexer convierte el texto fuente en una secuencia de tokens; el parser consume esos tokens, construye una representación estructural del programa y aplica validaciones de contexto; finalmente, main.py orquesta el proceso e informa al usuario del resultado.

3. Analizador léxico

El analizador léxico implementado en P1 se reutiliza sin modificaciones funcionales en esta entrega. Su objetivo es transformar la secuencia de caracteres de entrada en una secuencia de tokens que el parser puede consumir.

3.1. Palabras reservadas

El lexer mantiene un diccionario reserved que asocia cada palabra reservada del lenguaje con su nombre de token. La regla de identificadores consulta este diccionario para decidir si el lexema reconocido debe tratarse como ID o como una palabra reservada.

Token	Lexema
TRUE / FALSE	true / false
INT / FLOAT / CHAR / BOOLEAN	int / float / char / boolean
VOID	void
RETURN	return
IF / ELSE	if / else
DO / WHILE	do / while
BREAK	break
PRINT	print
NEW / RECORD	new / record

3.2. Tokens del lenguaje

Categoría	Tokens	Símbolo
Identificadores	ID	—
Literales	INT_VALUE, FLOAT_VALUE, CHAR_VALUE	—
Asignación/comparación	ASSIGN, EQ, GT, GE, LT, LE	= == > >= < <=
Aritméticos/lógicos	PLUS, MINUS, TIMES, DIVIDE, AND, OR, NOT	+ - * / && || !
Acceso a campo	DOT	.
Separadores	COMMA, SEMICOLON, LPAREN, RPAREN, LBRACE, RBRACE	, ; ( ) { }

3.3. Comentarios, literales y errores

Se soportan comentarios de línea (// hasta fin de línea) y de bloque (/* ... */). El de bloque actualiza el contador de líneas según los saltos de línea que contenga, evitando desajustes en los mensajes de error.

Los literales enteros aceptan notación decimal, binaria (0b), octal (0) y hexadecimal (0x con dígitos en mayúsculas). Los reales admiten punto decimal y notación científica (e±). Los caracteres se reconocen entre comillas simples con soporte de secuencias de escape.

Ante un carácter ilegal, el lexer registra un mensaje de error con línea en lexer.errors y avanza con lexer.skip(1), permitiendo continuar el análisis.

4. Analizador sintáctico

4.1. Representación estructural intermedia

Durante el parseo se construyen nodos mediante la función auxiliar node(kind, line, **data). Cada nodo es un diccionario con el tipo de construcción (kind), la línea de origen y los datos específicos de cada producción. Esta representación no es un AST completo de compilador, pero sí una estructura suficiente para realizar la validación adicional de restricciones de contexto que el enunciado exige y que una gramática libre de contexto pura no puede expresar cómodamente.

4.2. Símbolo inicial y estructura global

El símbolo inicial es program, que deriva en una lista opcional de elementos de programa program_items_opt. Esto permite aceptar ficheros vacíos, ficheros con solo saltos de línea y ficheros con solo «;», tal como especifica el enunciado.

Un program_item puede ser: un «;» aislado (ignorado), una declaración de registro, una función con retorno void/primitivo/de tipo registro, una declaración de variable con o sin inicialización, una estructura de control, o una sentencia simple terminada en «;».

Las declaraciones de funciones y de registros solo se permiten a nivel global (program_item), no dentro de bloques (block_item). Esta restricción se impone directamente en la gramática.

4.3. Separación entre primitive_type y custom_type

Una de las decisiones de diseño más importantes es distinguir en la gramática entre primitive_type (INT | FLOAT | CHAR | BOOLEAN) y custom_type (ID). Esta separación permite modelar la restricción central del enunciado: las variables de tipo registro deben declararse siempre con inicialización.

Para primitive_type se admite declaración simple, multideclaración e inicialización. Para custom_type solo existe la forma con ASSIGN. De este modo:

- int a, b; → válido (multideclaración primitiva)
- float x = 3.14; → válido (inicialización primitiva)
- Point p = new Point(1, 2); → válido (inicialización de registro)
- Point p; → ERROR sintáctico (declaración de registro sin inicialización)
- Point a, b; → ERROR sintáctico (multideclaración de registro)

Esta restricción se impone a nivel gramatical, sin comprobación semántica posterior.

4.4. Jerarquía de expresiones

La gramática de expresiones se organiza en tres niveles de no terminales:

- **primary_expr:** elementos atómicos: literales, identificadores, expresiones parentizadas y construcción con new.

- **postfix_expr:** expresiones primarias más llamadas a función (postfix_expr LPAREN arg_list_opt RPAREN) y acceso a campo (postfix_expr DOT ID). La recursión por la izquierda permite encadenar accesos y llamadas.
- **expression:** operaciones unarias (PLUS, MINUS, NOT) y binarias aritméticas, lógicas y comparativas.

Esta jerarquía garantiza que las llamadas a función y los accesos a campo tienen mayor precedencia que cualquier operador, sin necesidad de incluir DOT en la tabla de precedencia. Expresiones como p.x * 2 o f(a).campo se parsean correctamente.

4.5. Asignaciones y validación del lvalue

La regla de asignación es: assignment → postfix_expr ASSIGN expression. El lado izquierdo se parsea como postfix_expr para soportar accesos encadenados como p.velocity.x = 3.0. Sin embargo, no toda postfix_expr es un destino válido.

La validación se realiza tras el parseo mediante is_lvalue(expr), que acepta únicamente nodos de tipo id (identificador simple) y nodos de tipo field cuyo valor base también sea un lvalue. Con esto se rechazan:

- f() = 3; → destino inválido (llamada a función)
- (a + b) = 4; → destino inválido (expresión aritmética)
- new Circle(1) = 2; → destino inválido (instanciación)

4.6. Control de flujo

Los bloques son obligatorios en todas las estructuras de control (if, while, do-while). Esta decisión elimina por completo el problema del dangling else: nunca hay ambigüedad sobre a qué if pertenece un else cuando el cuerpo siempre está delimitado por llaves. No fue necesario el truco del token ficticio IFX.

La sentencia do-while sigue la forma DO block WHILE LPAREN expression RPAREN sin punto y coma final, tal como muestra la especificación.

4.7. Funciones y registros

El tipo de retorno void se trata de forma separada en la gramática para evitar que se pueda declarar una variable de tipo void. Si void se hubiera incluido dentro de primitive_type, la gramática habría aceptado declaraciones como void x;.

La declaración de registros sigue la forma record ID (field_list_opt) ; y solo aparece como program_item, nunca dentro de un bloque. La instanciación con new se modela como primary_expr, por lo que puede aparecer en cualquier contexto de expresión.

5. Definición formal de la gramática

Formalmente, la gramática implementada es una 4-tupla G = (Vn, Vt, P, S) donde S = program.

5.1. Símbolos no terminales (Vn)

```
program                 program_items_opt
program_item            record_decl
field_list_opt          field_list          field_decl
param_list_opt          param_list          param_decl
block                   block_items_opt     block_item
plain_simple_statement                      control_statement
primitive_id_list_tail                      comma_id_list
assignment
primitive_type          custom_type         any_type
expression              postfix_expr        primary_expr
literal
arg_list_opt            arg_list
empty
```

5.2. Producciones principales (P)

```
program → program_items_opt

program_items_opt → empty
                  | program_items_opt program_item

program_item → SEMICOLON
             | record_decl SEMICOLON
             | VOID ID LPAREN param_list_opt RPAREN block
             | primitive_type ID LPAREN param_list_opt RPAREN block
             | custom_type   ID LPAREN param_list_opt RPAREN block
             | primitive_type ID ASSIGN expression SEMICOLON
             | primitive_type ID primitive_id_list_tail SEMICOLON
             | custom_type   ID ASSIGN expression SEMICOLON
             | control_statement
             | plain_simple_statement SEMICOLON

record_decl → RECORD ID LPAREN field_list_opt RPAREN

field_list_opt → empty | field_list
field_list     → field_decl | field_list COMMA field_decl
field_decl     → any_type ID

param_list_opt → empty | param_list
param_list     → param_decl | param_list COMMA param_decl
param_decl     → any_type ID

block            → LBRACE block_items_opt RBRACE
block_items_opt  → empty | block_items_opt block_item
block_item       → SEMICOLON
                 | control_statement
                 | primitive_type ID ASSIGN expression SEMICOLON
                 | primitive_type ID primitive_id_list_tail SEMICOLON
                 | custom_type   ID ASSIGN expression SEMICOLON
                 | plain_simple_statement SEMICOLON

plain_simple_statement → assignment
                       | BREAK
                       | RETURN
```

```
                        | RETURN expression
                        | PRINT LPAREN expression RPAREN
                        | expression

control_statement → IF LPAREN expression RPAREN block
                  | IF LPAREN expression RPAREN block ELSE block
                  | WHILE LPAREN expression RPAREN block
                  | DO block WHILE LPAREN expression RPAREN

primitive_id_list_tail → empty | comma_id_list
comma_id_list          → COMMA ID | comma_id_list COMMA ID

assignment → postfix_expr ASSIGN expression

primitive_type → INT | FLOAT | CHAR | BOOLEAN
custom_type    → ID
any_type       → primitive_type | custom_type

expression → postfix_expr
           | PLUS  expression  %prec UPLUS
           | MINUS expression  %prec UMINUS
           | NOT   expression
           | expression TIMES  expression
           | expression DIVIDE expression
           | expression PLUS   expression
           | expression MINUS  expression
           | expression GT  expression
           | expression GE  expression
           | expression LT  expression
           | expression LE  expression
           | expression EQ  expression
           | expression AND expression
           | expression OR  expression

postfix_expr → primary_expr
             | postfix_expr LPAREN arg_list_opt RPAREN
             | postfix_expr DOT ID

primary_expr → literal
             | ID
             | LPAREN expression RPAREN
             | NEW ID LPAREN arg_list_opt RPAREN

literal → INT_VALUE | FLOAT_VALUE | CHAR_VALUE | TRUE | FALSE

arg_list_opt → empty | arg_list
arg_list     → expression | arg_list COMMA expression

empty → ε
```

6. Precedencia y asociatividad

El enunciado recomienda usar la herramienta de precedencia de PLY en lugar de estratificar la gramática en múltiples niveles de no terminales para operadores binarios. Esta implementación sigue esa recomendación.

6.1. Tabla de precedencia

Nivel	Token(s)	Asociatividad	Justificación
1 (menor)	OR	left	Disyunción lógica
2	AND	left	Conjunción lógica
3	EQ, GT, GE, LT, LE	nonassoc	Comparativos; evita a==b==c
4	PLUS, MINUS	left	Suma y resta binarias
5	TIMES, DIVIDE	left	Multiplicación y división
6 (mayor)	NOT, UPLUS, UMINUS	right	Operadores unarios

6.2. Operadores unarios UMINUS y UPLUS

El token MINUS aparece tanto como operador binario de sustracción como operador unario de negación. Para resolver la ambigüedad se declaran los pseudo-tokens UMINUS y UPLUS con mayor precedencia, y se anotan las reglas de los operadores unarios con %prec. Con esto, -a + b se interpreta como (-a) + b, que es lo correcto.

6.3. Precedencia de acceso a campo y llamadas: enfoque estructural

El acceso a campo (DOT) y las llamadas a función no se incluyen en la tabla de precedencia. Su precedencia máxima se garantiza estructuralmente mediante la jerarquía primary_expr → postfix_expr → expression: las operaciones postfijas siempre se reducen antes que cualquier operador binario o unario.

6.4. Ausencia del conflicto dangling else

En esta implementación no existe el conflicto shift/reduce del dangling else. La razón es que todas las ramas de control de flujo exigen un bloque delimitado por llaves, no una sentencia desnuda. Por tanto, no hay ambigüedad posible sobre a qué if pertenece un else. No fue necesario el truco del token ficticio IFX.

7. Validaciones adicionales orientadas a P2

El parser no se limita a aceptar o rechazar según la gramática libre de contexto, sino que realiza una validación estructural sobre los nodos construidos durante el parseo. La función central es validate_program(items), que itera sobre los elementos del programa y despacha la validación de cada función a validate_function() y de cada sentencia a validate_item().

7.1. Validez del lado izquierdo de una asignación

La función is_lvalue(expr) comprueba si un nodo es un destino de asignación válido. Solo acepta nodos de tipo id (identificador simple) y nodos de tipo field cuyo base también sea un lvalue. Cualquier otra expresión genera:

```
[ERROR] Lado izquierdo de asignación inválido en la línea N
```

7.2. Uso correcto de break

validate_block() propaga el flag in_loop al descender en bucles (while, do-while), pero no al descender en bloques if. Cuando se encuentra un nodo break con in_loop = False:

```
[ERROR] 'break' fuera de bucle en la línea N
```

7.3. Uso correcto de return

Al validar un nodo return se comprueban tres condiciones:

- return fuera de función → [ERROR] 'return' fuera de función en la línea N
- return en función void → [ERROR] 'return' no permitido en función void en la línea N
- return sin expresión en función no void → [ERROR] 'return' sin expresión en función no void en la línea N

7.4. Presencia obligatoria de return en funciones no void

validate_function() mantiene un estado has_value_return que se activa cuando se encuentra un return con expresión dentro del cuerpo. Si al terminar el recorrido el flag sigue en falso para una función no void:

```
[ERROR] La función 'nombre' debe contener al menos un return con expresión en la línea N
```

Esta comprobación no intenta análisis de flujo exhaustivo, sino que verifica la presencia estructural de al menos un return con valor. Para P2 esta aproximación es razonable y defendible.

7.5. Acumulación y reporte de errores

parse_text(text) combina en una sola lista los errores léxicos (lexer.errors), los errores de parseo (_parse_errors) y los errores de validación (validate_program()). Si la lista está vacía el programa es aceptado; en caso contrario se imprimen los errores y se devuelve código de salida 1.

8. Punto de entrada y modos de ejecución

8.1. Modo normal

Ejecuta el parser completo sobre el fichero de entrada:

```
python3 main.py fichero.lava
```

Si el programa es válido: no produce salida y termina con código 0. Si hay errores: imprime cada mensaje y termina con código 1.

8.2. Modo --token

Ejecuta únicamente el lexer y genera un fichero .token:

```
python3 main.py --token fichero.lava
```

El fichero generado sigue el formato heredado de P1: TIPO, VALOR, LINEA, COLUMNA_INICIO, COLUMNA_FIN. Mantener este modo permite seguir validando el lexer de forma independiente.

8.3. Integración léxico-parser

El objeto lexer se pasa directamente al método parse(), de modo que el parser llama al lexer token a token bajo demanda. El flujo es completamente on-the-fly, que es exactamente la forma de trabajo que PLY espera.

9. Batería de pruebas

Se ha desarrollado una batería de 22 casos de prueba organizada en tests/valid/ (9 casos), tests/invalid/ (13 casos) y tests/expected_invalid/ (salidas esperadas). El script run_tests.py automatiza la ejecución completa:

```
cd P2_MARCO_GARCIA_PABLO_ROIG/tests
python3 run_tests.py
# Resultado esperado: Summary: 22/22 passed
```

9.1. Casos válidos

Caso	Descripción
01_newlines_only.lava	Fichero con solo saltos de línea
02_semicolons_only.lava	Fichero con solo «;»
03_primitive_decls.lava	Declaraciones simples y múltiples, asignaciones, print
04_records_access.lava	Registro, new y acceso por punto
05_control_flow.lava	do-while, while, if anidado con break
06_functions_calls.lava	Función con parámetros, llamada, uso del retorno
07_record_function_return.lava	Función que devuelve un registro, new como retorno
08_nested_fields.lava	Registros anidados y acceso encadenado por punto
09_precedence_parentheses.lava	Precedencia de operadores y paréntesis

9.2. Casos inválidos

Caso	Error detectado
01_invalid_assignment_call.lava	f() = 3; — lvalue inválido (llamada a función)
02_invalid_assignment_expr.lava	(a+b) = 3; — lvalue inválido (expresión)
03_invalid_record_without_init.lava	Point p; — registro sin inicialización
04_invalid_record_multidecl.lava	Point a, b; — multideclaración de registro
05_invalid_break_top.lava	break; a nivel global
06_invalid_break_in_if.lava	break; dentro de if sin bucle exterior
07_invalid_return_top.lava	return 1; a nivel global
08_invalid_void_return.lava	return expr; en función void
09_invalid_nonvoid_missing_return.lava	Función no void sin ningún return
10_invalid_nonvoid_empty_return.lava	return; sin expresión en función no void
11_invalid_multidecl_with_assign.lava	float d, e = 0; — multidecl con asignación
12_invalid_unexpected_eof.lava	Fin de archivo inesperado
13_invalid_chained_assignment.lava	Asignación encadenada inválida

10. Decisiones de diseño y problemas encontrados

10.1. Separación primitive_type / custom_type

Una alternativa más simple habría sido definir un único no terminal type que englobara todos los tipos. Sin embargo, eso habría obligado a tratar la restricción de inicialización obligatoria de registros en la fase semántica. Al separar explícitamente ambos tipos en la gramática, la restricción se impone directamente en las producciones de program_item y block_item: para custom_type solo existe la alternativa con ASSIGN.

10.2. Jerarquía de tres niveles de expresiones

En lugar de una única regla expr plana con la tabla de precedencia para todos los operadores, se organizan las expresiones en primary_expr → postfix_expr → expression. Esto es más natural para modelar que las llamadas a función y los accesos a campo tienen precedencia máxima y se encadenan recursivamente. Además simplifica la validación de lvalues, ya que los nodos field siempre tienen una estructura predecible.

10.3. Ausencia de conflictos shift/reduce

Gracias a las decisiones de diseño adoptadas, el parser genera las tablas LALR sin ningún warning ni conflicto: los bloques obligatorios eliminan el dangling else; UMINUS/UPLUS resuelven el doble papel de PLUS/MINUS; la jerarquía de expresiones elimina la necesidad de incluir DOT en la tabla de precedencia; y la separación de tipos elimina ambigüedades en las declaraciones.

10.4. Validación estructural post-parseo vs. gramática más compleja

Las restricciones de contexto (break en bucle, return según tipo de función) podrían intentarse expresar puramente en la gramática libre de contexto. Sin embargo, el coste sería inaceptable: habría que duplicar todos los no terminales de bloques y sentencias en versiones distintas según el contexto. La alternativa adoptada es mucho más limpia y produce mensajes de error informativos con número de línea.

10.5. Multideclaración con asignación

La gramática rechaza float d, e = 0xFF; de forma natural porque la producción primitive_type ID primitive_id_list_tail SEMICOLON construye solo una lista de identificadores sin expresiones de inicialización, y la producción con ASSIGN solo acepta un único identificador. No existe ninguna producción que combine ambas formas.

11. Uso de inteligencia artificial generativa

De acuerdo con la normativa de la asignatura, declaramos el uso de herramientas de IA generativa (Claude y Gemini) durante el desarrollo de la práctica. Hemos utilizado la IA principalmente como apoyo para investigar patrones de diseño en construcción de parsers con PLY cuando encontramos enfoques que producían conflictos o estructuras demasiado repetitivas, para depurar conflictos de la gramática y para estructurar la memoria.

Todo el código resultante ha sido analizado, adaptado a nuestra estructura y comprendido en su totalidad. Las decisiones de diseño reflejadas tanto en el código como en esta memoria son nuestras.

12. Conclusiones

El desarrollo del analizador sintáctico ha sido la fase más exigente del proyecto, especialmente en lo que respecta al diseño de la gramática y a la elección de qué restricciones imponer a nivel gramatical frente a cuáles delegar a una validación posterior.

La gramática resultante cubre la totalidad de las construcciones del lenguaje Lava y genera las tablas LALR sin ningún conflicto ni warning. La separación entre tipos primitivos y tipos de registro, la jerarquía de expresiones en tres niveles y la validación estructural post-parseo son los tres pilares que permiten rechazar correctamente todos los casos inválidos sin necesitar análisis semántico completo.

La batería de 22 pruebas verifica de forma automática tanto los casos válidos como los casos límite inválidos, aportando evidencia objetiva del correcto funcionamiento del parser.

El mayor aprendizaje de esta entrega ha sido entender que una gramática no es solo una descripción formal de un lenguaje, sino también un mecanismo de control: diseñar bien las producciones permite rechazar código incorrecto sin necesitar análisis semántico, simplificando enormemente las fases posteriores del compilador.